SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1102

COURSE OUTCOME COVERED

CO4:
implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS *(BL5)*

Assessment Tool Weightage: 30%

QUESTIONS: 6 **Total Marks:30**

Annexure A

Reverse Engineering Task

Code of Conduct:

*I **NIRANJANA (192521370) (**Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: NIRANJANA A

Annexure A

Reverse Engineering Task

Object-Oriented Reverse Engineering and Design Patterns

1. Legacy E-Learning Management System

Introduction

- The existing E-Learning Management System contains large modules responsible for student registration, course management, content delivery, assignment evaluation, and result generation. The user interface code and business rules are tightly coupled, making the system difficult to modify, test, and maintain.
- The system can be reverse engineered by identifying the appropriate classes, attributes, methods, responsibilities, and relationships. It can then be redesigned using object-oriented principles and layered architecture.
    - Reverse Engineering of the Existing System

The major classes identified from the existing system are:

- Student Class
- Attributes:
- studentId
- name
- email
- password
- Methods:
- register()
- login()
- enrollCourse()
- submitAssignment()
- viewResult()

Responsibility:

The Student class represents a learner and manages student-related information and activities.

Course Class

Attributes:

- courseId
- courseName
- description
- duration

Methods:

- addCourse()
- updateCourse()
- addContent()
- addAssignment()

Responsibility:

The Course class manages course information and its associated learning materials.

Content Class

Attributes:

- contentId
- title
- contentType
- resourceURL

Methods:

- uploadContent()
- updateContent()
- deliverContent()

Responsibility:

The Content class manages learning materials such as videos, documents, and presentations.

Assignment Class

Attributes:

- assignmentId
- title
- deadline
- maximumMarks

Methods:

- createAssignment()
- submitAssignment()
- evaluateAssignment()

Responsibility:

The Assignment class manages assignment creation, submission, and evaluation.

Submission Class

Attributes:

- submissionId
- submissionDate
- marks
- feedback

Methods:

- submit()
- calculateMarks()
- provideFeedback()

Responsibility:

The Submission class represents a student's submitted assignment.

Result Class

Attributes:

- resultId
- marks
- grade
- percentage

Methods:

- calculateResult()
- generateGrade()
- displayResult()

Responsibility:

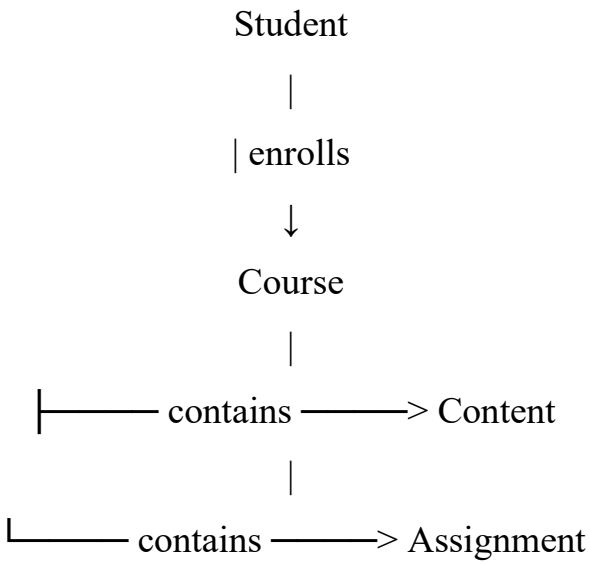
The Result class manages student academic results.

1.2 Relationships Between Classes

The main relationships are:

- A Student enrolls in one or more Courses.
- A Course contains multiple Content objects.
- A Course contains multiple Assignments.
- A Student submits Assignments.
- An Assignment can have multiple Submissions.
- A Student receives Results.
- A Course is associated with multiple Students.

The relationships can be represented as:



|

↓

Submission

|

↓

Result

1.3 Problems in the Legacy System

The existing system has the following design problems:

1. User interface and business logic are tightly coupled.
2. Large classes perform multiple responsibilities.
3. Database operations may be mixed with business logic.
4. Changes in one module can affect several other modules.
5. Code duplication may occur.
6. Testing individual components becomes difficult.
7. The system has low cohesion and high coupling.

1.4 Redesigned Layered Architecture

The system can be redesigned using a layered architecture:

Presentation Layer

↓

Service / Business Layer

↓

Domain Layer

↓

Data Access Layer

↓

Database

Presentation Layer

The Presentation Layer handles the user interface. It provides screens for registration, login, course enrollment, content viewing, assignment submission, and result viewing.

It should not contain the main business rules.

Service Layer

The Service Layer contains application and business operations.

Examples include:

- StudentService
- CourseService
- AssignmentService
- ResultService

For example, CourseService handles course enrollment and course-related operations.

Domain Layer

The Domain Layer contains the core business objects:

- Student
- Course
- Content
- Assignment
- Submission
- Result

These classes represent the real-world entities of the e-learning system.

Data Access Layer

The Data Access Layer is responsible for database operations.

Examples include:

- StudentRepository
- CourseRepository
- AssignmentRepository
- ResultRepository

The Service Layer communicates with the repositories instead of directly accessing the database.

1.5 Object-Oriented Principles

Encapsulation

Student, course, and result data are kept inside their respective classes and accessed through appropriate methods.

Abstraction

Complex operations such as database access and assignment evaluation are hidden behind service and repository interfaces.

Inheritance

Common functionality can be placed in a base class and reused by specialized classes.

Polymorphism

Different types of assignments or learning content can provide their own implementations of common operations.

1.6 Improvements

Cohesion

Each class has a focused responsibility. For example, Student manages student information while Course manages course information.

Reduced Coupling

The Presentation Layer does not directly depend on database implementation. It communicates through the Service Layer.

Maintainability

Changes to the user interface can be made without modifying business logic.

Scalability

New features such as online examinations, certificates, quizzes, and discussion forums can be added as separate components.

Conclusion

The redesigned E-Learning Management System provides better cohesion, lower coupling, improved maintainability, and scalability. Object-oriented principles and layered architecture separate responsibilities and make the system easier to modify and extend.

2. Legacy Customer Support System

Introduction

- The existing Customer Support System uses multiple conditional statements to process support requests through email, chatbot, telephone, and social media. Adding a new support channel requires modifications to several existing modules.
- This indicates poor object-oriented design and high coupling. The system can be reverse engineered and redesigned using a suitable behavioral design pattern, particularly the Strategy Pattern.

2.1 Problems in the Existing System

The existing implementation may contain code such as:

```
if channel = Email
   handleEmail()

else if channel = Chatbot
   handleChatbot()

else if channel = Telephone
   handleTelephone()

else if channel = SocialMedia
   handleSocialMedia()
```

This creates several problems:

1. Large conditional statements.

2. High coupling between the support manager and channels.
3. Difficult addition of new channels.
4. Violation of the Open/Closed Principle.
5. Difficult testing.
6. Poor maintainability.
7. Existing modules must be modified whenever a new channel is introduced.

2.2 Reverse-Engineered Classes

SupportRequest Class

Attributes:

- requestId
- customerId
- message
- priority

Methods:

- createRequest()
- updateRequest()
- getRequestDetails()

SupportChannel Interface

Method:

- handleRequest()

This interface defines a common operation for all support channels.

EmailSupport Class

Method:

- handleRequest()

Responsibility:

Handles customer requests received through email.

ChatbotSupport Class

Method:

- handleRequest()

Responsibility:

Processes requests received through the chatbot.

TelephoneSupport Class

Method:

- handleRequest()

Responsibility:

Handles telephone-based support requests.

SocialMediaSupport Class

Method:

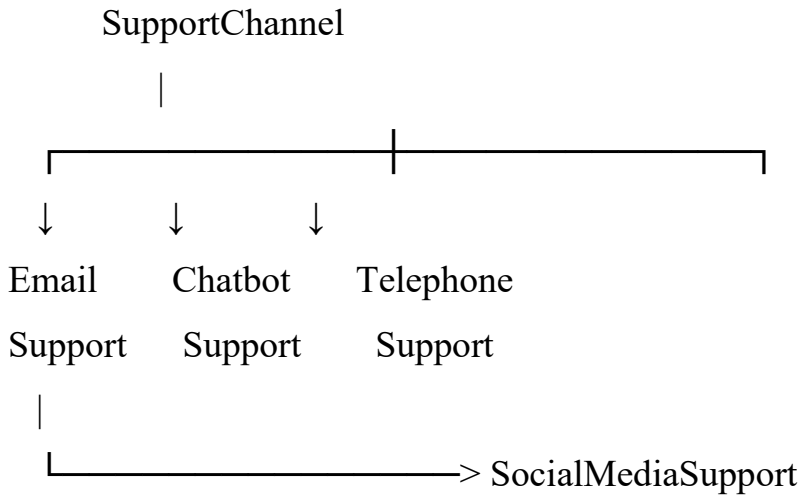
- handleRequest()

Responsibility:

Processes requests received through social media.

2.3 Redesigned Structure Using Strategy Pattern

The Strategy Pattern can be used to encapsulate different support-channel behaviors.



The SupportChannel interface defines:

handleRequest()

Each concrete support channel implements this method differently.

2.4 Support Manager

A SupportManager class can use the SupportChannel abstraction.

SupportManager

↓

SupportChannel

↓

EmailSupport / ChatbotSupport /

TelephoneSupport / SocialMediaSupport

The Support Manager does not need to know the internal implementation of each channel.

2.5 Adding a New Channel

Suppose WhatsApp support is required.

A new class can be created:

WhatsAppSupport implements SupportChannel

The existing EmailSupport, ChatbotSupport, TelephoneSupport, and SocialMediaSupport classes do not need to be modified.

2.6 Benefits of the Redesigned System

Flexibility

The support channel can be changed dynamically according to the requirement.

Extensibility

New channels such as WhatsApp, mobile applications, or web support can be added easily.

Loose Coupling

The SupportManager depends on the SupportChannel abstraction rather than concrete implementations.

Maintainability

Each channel has its own class and responsibility, making the code easier to understand and modify.

Open/Closed Principle

The system is open for extension but closed for modification.

Testability

Each support-channel implementation can be tested independently.

Conclusion

The Strategy Pattern removes large conditional statements and separates support-channel behavior into independent classes. The redesigned system provides flexibility, extensibility, loose coupling, and improved maintainability.

3. Legacy Banking Transaction System

Introduction

- The existing Banking Transaction System contains account management, transaction validation, fund transfer, loan processing, and transaction reporting in a few tightly coupled classes. Database operations are also directly embedded inside business methods.
- This results in poor separation of concerns, high coupling, low cohesion, and difficult testing.

3.1 Reverse-Engineered Domain Classes

Customer Class

Attributes:

- customerId
- name
- address
- contactNumber

Methods:

- createAccount()
- updateProfile()
- viewAccounts()

Account Class

Attributes:

- accountId
- accountNumber

- balance
- accountType

Methods:

- deposit()
- withdraw()
- checkBalance()

Transaction Class

Attributes:

- transactionId
- transactionType
- amount
- transactionDate

Methods:

- validateTransaction()
- processTransaction()

Loan Class

Attributes:

- loanId
- loanAmount
- interestRate
- loanStatus

Methods:

- applyLoan()
- calculateInterest()
- approveLoan()

Report Class

Attributes:

- reportId
- reportType

- generatedDate

Methods:

- generateReport()
- displayReport()

3.2 Service Classes

Business operations should be separated into service classes.

AccountService

Responsible for account creation, deposits, withdrawals, and account management.

TransactionService

Responsible for transaction validation and processing.

FundTransferService

Responsible for transferring funds between accounts.

LoanService

Responsible for loan application, validation, approval, and interest calculation.

ReportService

Responsible for generating transaction reports.

3.3 Data Access Components

Database operations should be moved into separate repositories or DAO classes.

Examples:

- AccountRepository
- TransactionRepository
- LoanRepository
- CustomerRepository

These components communicate with the database.

3.4 Layered Architecture

Presentation Layer

↓

Service Layer

↓

Domain Layer

↓

Repository / DAO Layer

↓

Database

Presentation Layer

Provides interfaces for account management, fund transfers, loans, and reports.

Service Layer

Contains business rules and transaction-processing logic.

Domain Layer

Contains Customer, Account, Transaction, Loan, and other domain objects.

Data Access Layer

Handles persistence using repositories or DAOs.

3.5 Repository Pattern

The Repository Pattern can be used to separate business logic from database operations.

For example:

FundTransferService

↓

AccountRepository

↓

Database

The FundTransferService does not directly execute database queries.

3.6 Separation of Concerns

The redesigned architecture ensures that:

- Presentation handles user interaction.
- Services handle business logic.
- Domain classes represent business entities.

- Repositories handle persistence.

3.7 Benefits

Separation of Concerns

Each layer performs a specific responsibility.

Testability

Services can be tested independently by using mock repositories.

Maintainability

Database implementation can be changed without modifying business logic.

Low Coupling

Services depend on repository interfaces rather than concrete database implementations.

High Cohesion

Each service contains closely related operations.

Conclusion

The redesigned Banking Transaction System separates domain logic, business services, and data access. The use of layered architecture and Repository/DAO patterns improves separation of concerns, testability, maintainability, and scalability.

4. Legacy Hotel Reservation System

Introduction

- The existing Hotel Reservation System directly performs OODBMS operations from reservation screens. Database queries for rooms, guests, bookings, and payments are repeated across multiple modules.
- This results in high coupling, code duplication, and poor persistence management. The system can be redesigned using the DAO Pattern or Repository Pattern.

4.1 Problems in the Existing Architecture

High Coupling

The reservation screen directly communicates with the OODBMS.

Reservation Screen

↓

OODBMS

Therefore, changes to the database structure can affect the user interface.

Code Duplication

The same database operations for rooms, guests, reservations, and payments may be repeated in different modules.

Poor Persistence Management

Database operations are mixed with user-interface and business logic.

Difficult Maintenance

Any database change requires modifications in multiple modules.

4.2 Reverse-Engineered Classes

Guest Class

Attributes:

- guestId
- name
- contact
- email

Methods:

- registerGuest()
- updateGuest()
- getGuestDetails()

Room Class

Attributes:

- roomId
- roomNumber
- roomType
- price

- availability

Methods:

- checkAvailability()
- updateRoom()
- calculatePrice()

Reservation Class

Attributes:

- reservationId
- guestId
- roomId
- checkInDate
- checkOutDate
- status

Methods:

- createReservation()
- cancelReservation()
- modifyReservation()

Payment Class

Attributes:

- paymentId
- amount
- paymentDate
- paymentStatus

Methods:

- makePayment()
- refundPayment()
- verifyPayment()

4.3 DAO Pattern

DAO stands for Data Access Object.

Separate DAO classes can be created:

GuestDAO

RoomDAO

ReservationDAO

PaymentDAO

Each DAO is responsible for persistence operations related to one type of object.

For example:

RoomDAO

├── saveRoom()

├── findRoom()

├── updateRoom()

├── deleteRoom()

└── findAvailableRooms()

4.4 Repository Pattern

A Repository can also be used to provide an abstraction for persistent domain objects.

ReservationService

↓

ReservationRepository

↓

OODBMS

The Repository hides persistence details from the business layer.

4.5 Redesigned Architecture

Reservation UI

↓

Reservation Service

↓

Domain Objects

↓

DAO / Repository

↓

OODBMS

Presentation Layer

Handles reservation screens, guest registration, check-in, check-out, and payment interfaces.

Service Layer

Handles business operations such as room availability, booking confirmation, cancellation, and payment processing.

Domain Layer

Contains Guest, Room, Reservation, and Payment objects.

Data Access Layer

Contains DAO or Repository classes that communicate with the OODBMS.

4.6 OODBMS Integration

The OODBMS stores persistent objects such as Room, Guest, Reservation, and Payment.

For example, when a user searches for available rooms:

User

↓

Reservation UI

↓

ReservationService

↓

RoomRepository

↓

OODBMS

The Repository retrieves the required Room objects and returns them to the Service Layer.

When a reservation is created:

ReservationService

↓

ReservationRepository

↓

OODBMS

The new Reservation object is stored as a persistent object.

4.7 Benefits

Modularity

Database operations are isolated in DAO or Repository classes.

Reusability

The same DAO methods can be used by different services.

Reduced Duplication

Common database operations are implemented only once.

Loose Coupling

The user interface does not directly depend on the OODBMS.

Maintainability

Database changes are isolated to the Data Access Layer.

OODBMS Integration

The Repository/DAO layer provides a clean mechanism for storing and retrieving persistent objects.

Conclusion

The redesigned Hotel Reservation System uses DAO/Repository patterns to separate persistence logic from presentation and business logic. This reduces coupling and duplication while improving modularity, reusability, maintainability, and OODBMS integration.

5. Legacy Transportation Management System

Introduction

- The existing Transportation Management System contains separate modules for buses, trains, taxis, and rental vehicles. Object creation logic is duplicated throughout the application.

For example, different modules may directly create objects using:

new Bus()

new Train()

new Taxi()

new RentalVehicle()

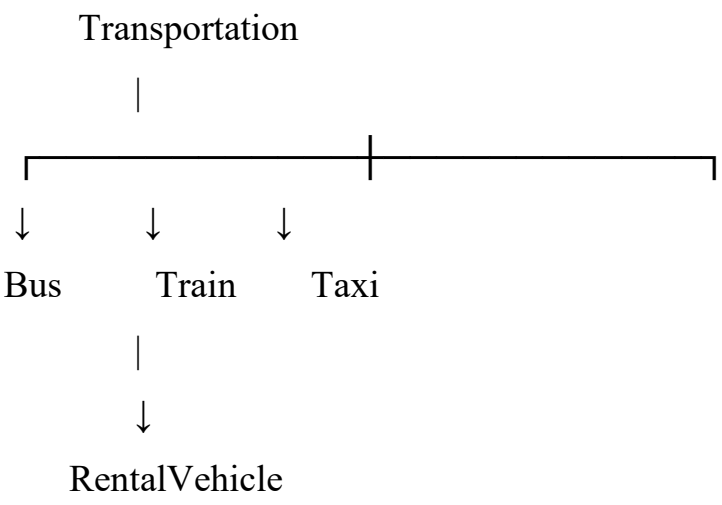
This creates high coupling and makes it difficult to introduce new transportation services.

5.1 Problems in the Existing System

1. Object creation logic is duplicated.
2. Client classes depend directly on concrete transportation classes.
3. Adding a new vehicle type requires changes in multiple modules.
4. High coupling exists between clients and concrete classes.
5. Code reuse is reduced.
6. Maintenance becomes difficult.

5.2 Common Abstraction

A common Transportation abstraction can be created.



Transportation Class

Attributes:

- vehicleId
- capacity
- route
- fare

Methods:

- book()
- calculateFare()
- getDetails()

The concrete transportation classes implement or inherit these common operations.

5.3 Factory Method Pattern

The Factory Method Pattern can be used to centralize object creation.

TransportationFactory

↓

createTransportation()

↓

Bus / Train / Taxi / RentalVehicle

Instead of the client directly creating objects, it requests the required transportation object from the factory.

For example:

Client

↓

TransportationFactory

↓

Transportation

↓

Bus / Train / Taxi

5.4 Concrete Factories

Different factories can be used if required:

BusFactory

TrainFactory

TaxiFactory

RentalVehicleFactory

Each factory creates its corresponding transportation object.

5.5 Abstract Factory Pattern

If the system needs to create families of related objects, the Abstract Factory Pattern can be used.

For example:

TransportationFactory

```
    |
    ├── createVehicle()
    ├── createTicket()
    └── createBooking()
```

Different concrete factories can create related objects for buses, trains, taxis, or rental services.

5.6 Object Relationships

The client depends on the common Transportation abstraction rather than concrete classes.

Client

↓

Transportation

↑

Bus

Train

Taxi

RentalVehicle

This reduces direct dependency on concrete implementations.

5.7 Benefits

Extensibility

New transportation types can be introduced by creating new classes and factory implementations.

Scalability

The system can support future services such as electric vehicles, metro services, or autonomous transportation.

Code Reuse

Common transportation behavior is defined in the common abstraction.

Loose Coupling

The client depends on the Transportation abstraction rather than concrete classes.

Maintainability

Object creation logic is centralized in factory classes.

Flexibility

The appropriate transportation object can be created dynamically according to user requirements.

Conclusion

The Factory Method or Abstract Factory Pattern provides a flexible solution for transportation object creation. By separating object creation from client code, the redesigned system improves extensibility, scalability, code reuse, and maintainability.

6. Legacy Healthcare Monitoring System

Introduction

- The existing Healthcare Monitoring System collects patient information, processes sensor readings, generates alerts, stores medical records, and produces reports. Several classes perform multiple unrelated tasks and

directly depend on concrete monitoring devices and storage components.This design violates important SOLID principles, particularly the Single Responsibility Principle (SRP), Open/Closed Principle (OCP), and Dependency Inversion Principle (DIP).

6.1 Problems in the Existing System

A legacy class may perform all of the following operations:

HealthcareMonitor

```
├── collectSensorData()
├── processSensorData()
├── generateAlert()
├── saveMedicalRecord()
└── generateReport()
```

This class has too many responsibilities.

It may also directly depend on concrete classes:

HealthcareMonitor

```
    ↓
```

ECGSensor

HeartRateSensor

Database

This creates tight coupling.

6.2 Single Responsibility Principle (SRP) Violation

The Single Responsibility Principle states that a class should have one main responsibility.

The legacy HealthcareMonitor class performs:

- Sensor data collection.
- Data processing.
- Alert generation.
- Medical record storage.
- Report generation.

These responsibilities should be separated into different classes.

6.3 Open/Closed Principle (OCP) Violation

The Open/Closed Principle states that software entities should be open for extension but closed for modification.

If a new sensor such as a blood-pressure sensor is introduced, the existing monitoring class should not require major modifications.

Instead, a common Sensor interface can be created.

6.4 Dependency Inversion Principle (DIP) Violation

The Dependency Inversion Principle states that high-level modules should depend on abstractions rather than concrete implementations.

The legacy system directly depends on concrete sensors and databases.

Instead of:

MonitoringSystem

↓

ECGSensor

the redesigned system uses:

MonitoringSystem

↓

Sensor Interface

↑

ECGSensor

HeartRateSensor

TemperatureSensor

Similarly, the monitoring system should not directly depend on a particular database.

6.5 Redesigned Classes

Patient Class

Attributes:

- patientId

- name
- age
- medicalHistory

Methods:

- updatePatientDetails()
- viewMedicalHistory()

Responsibility:

Represents patient information.

Sensor Interface

Methods:

- readData()
- getSensorType()

Responsibility:

Defines common operations for all monitoring devices.

ECGSensor Class

Methods:

- readData()
- getSensorType()

Responsibility:

Collects ECG readings.

HeartRateSensor Class

Methods:

- readData()
- getSensorType()

Responsibility:

Collects heart-rate readings.

TemperatureSensor Class

Methods:

- readData()

- getSensorType()

Responsibility:

Collects body-temperature readings.

SensorDataProcessor Class

Methods:

- processReading()
- validateReading()
- analyzeReading()

Responsibility:

Processes and analyzes sensor readings.

AlertService Class

Methods:

- generateAlert()
- sendAlert()

Responsibility:

Generates alerts when abnormal readings are detected.

MedicalRecordRepository Interface

Methods:

- saveRecord()
- findRecord()
- updateRecord()

Responsibility:

Defines operations for storing and retrieving medical records.

ReportService Class

Methods:

- generateReport()
- generatePatientReport()

Responsibility:

Generates medical reports from processed information.

6.6 Redesigned Architecture

The redesigned system can be represented as:

Patient / Sensors

↓

Monitoring Service

↓

Sensor Data Processor

↓

Alert Service

↓

Medical Record Repository

↓

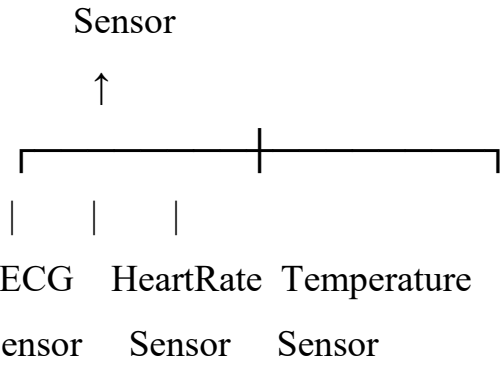
Database

↓

Report Service

6.7 Use of Interfaces and Abstraction

The system uses the Sensor interface as an abstraction.



The Monitoring Service depends only on the Sensor interface.

Therefore, a new sensor can be added without modifying the Monitoring Service.

Similarly:

Monitoring Service

↓

MedicalRecordRepository

↑

DatabaseRepository

The high-level business logic depends on the repository abstraction rather than a concrete database implementation.

6.8 Design Patterns

Strategy Pattern

The Strategy Pattern can be used for different methods of analyzing sensor readings.

AnalysisStrategy

↓

NormalAnalysis

AbnormalityDetection

CriticalConditionAnalysis

Different analysis strategies can be selected according to the monitoring requirement.

Observer Pattern

The Observer Pattern can be used for alerts.

Sensor Monitoring

↓

Observer

↙ ↘

Doctor Patient

When an abnormal reading is detected, registered observers can be automatically notified.

Repository Pattern

The Repository Pattern separates medical record persistence from business logic.

MedicalRecordService

↓

MedicalRecordRepository

↓

Database

6.9 SOLID Improvements

SRP Improvement

Each class now has a single responsibility. Sensor classes collect data, the processor analyzes data, the AlertService generates alerts, and the Repository handles persistence.

OCP Improvement

New sensors and analysis methods can be added by creating new implementations rather than modifying existing classes.

DIP Improvement

High-level services depend on interfaces such as Sensor and MedicalRecordRepository.

6.10 Benefits

High Cohesion

Each class performs closely related operations.

Loose Coupling

The monitoring system is not directly dependent on concrete sensor or database implementations.

Flexibility

Different sensors, analysis methods, and storage implementations can be replaced easily.

Testability

Interfaces allow mock sensors and mock repositories to be used during testing.

Maintainability

Changes are isolated to individual classes and components.

Extensibility

New sensors, alert mechanisms, and medical analysis algorithms can be added without major changes to the existing system.

Conclusion

The redesigned Healthcare Monitoring System follows SOLID principles by separating responsibilities, using abstractions, and reducing dependencies on concrete implementations. The use of interfaces, Strategy, Observer, and Repository patterns improves cohesion, loose coupling, flexibility, testability, extensibility, and maintainability.

RUBRICS FOR CALCULATION

Reverse Engineering Task

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation